

ORM API

Models

Model fields are defined as attributes on the model itself:

```
from odoo import models, fields
class AModel(models.Model):
    _name = 'a.model.name'

    field1 = fields.Char()
```

Warning

this means you cannot define a field and a method with the same name, the last one will silently overwrite the former ones.

By default, the field's label (user-visible name) is a capitalized version of the field name, this can be overridden with the `string` parameter.

```
field2 = fields.Integer(string="Field Label")
```

For the list of field types and parameters, see [the fields reference](#).

Default values are defined as parameters on fields, either as a value:

```
name = fields.Char(default="a value")
```

or as a function called to compute the default value, which should return that value:

```
def _default_name(self):
    return self.get_value()

name = fields.Char(default=lambda self: self._default_name())
```

Odoo models are created by inheriting one of the following.

- **Model** for regular database-persisted models
- **TransientModel** for temporary data, stored in the database but automatically vacuumed every so often
- **AbstractModel** for abstract super classes meant to be shared by multiple inheriting models

The system automatically instantiates every model once per database. Those instances represent the available models on each database, and depend on which modules are installed on that database. The actual class of each instance is built from the Python classes that create and inherit from the corresponding model.

Every model instance is a "recordset", i.e., an ordered collection of records of the model. Recordsets are returned by methods like `browse()`, `search()`, or field accesses. Records have no explicit representation: a record is represented as a recordset of one record.

To create a class that should not be instantiated, the `_register` attribute may be set to `False`.

`_auto` : `bool` [↗](#) = `False`

Whether a database table should be created. If set to `False`, override `init()` to create the database table.

Automatically defaults to `True` for abstract models.

Tip

To create a model without any table, inherit from `AbstractModel`.

`_log_access`

Whether the ORM should automatically generate and update the **Access Log fields**.

Defaults to whatever value was set for `_auto`.

`_table` : `str` [↗](#) = ''

SQL table name used by model if `_auto`

`_register` : `bool` [↗](#) = `False`

registry visibility

`_abstract` : `bool` [↗](#) = `True`

Whether the model is *abstract*.

`_transient : bool = False`

Whether the model is *transient*.

See also

`TransientModel`

`_name : str = None`

the model name (in dot-notation, module namespace)

`_description : str | None = None`

the model's informal name

`_inherit : str | List[str] | tuple[str, ...] = ()`

Python-inherited models:

Type:

`str` or `list(str)` or `tuple(str, ...)`

Note

- If `_name` is set, name(s) of parent models to inherit from
- If `_name` is unset, name of a single model to extend in-place

`_inherits : frozendict[str, str] = {}`

dictionary {'parent_model': 'm2o_field'} mapping the `_name` of the parent business objects to the names of the corresponding foreign key fields to use:

```
_inherits = {
    'a.model': 'a_field_id',
    'b.model': 'b_field_id'
}
```

implements composition-based inheritance: the new model exposes all the fields of the inherited models but stores none of them: the values themselves remain stored on the linked record.

Warning

`_rec_name : str | None = None`

field to use for labeling records, default: name

`_order : str = 'id'`

default order field for searching results

`_check_company_auto : bool = False`

On write and create, call `_check_company` to ensure companies consistency on the relational fields having `check_company=True` as attribute.

`_parent_name : str = 'parent_id'`

the many2one field used as parent field

`_parent_store : bool = False`

set to True to compute `parent_path` field.

Alongside a `parent_path` field, sets up an indexed storage of the tree structure of records, to enable faster hierarchical queries on the records of the current model using the `child_of` and `parent_of` domain operators.

`_fold_name : str = 'fold'`

field to determine folded groups in kanban views

AbstractModel

`odoo.models.AbstractModel` [\[source\]](#)

alias of `odoo.orm.models.BaseModel`

Model

`class odoo.models.Model (env: Environment, ids: tuple[IdType, ...],
prefetch_ids: Reversible[IdType])` [\[source\]](#)

Main super-class for regular database-persisted Odoo models.

Odoo models are created by inheriting from this class:

```
class ResUsers(Model):
    ...
```

Whether a database table should be created. If set to `False`, override `init()` to create the database table.

Automatically defaults to `True` for abstract models.

Tip

To create a model without any table, inherit from `AbstractModel`.

```
_abstract : typing.Literal[False] = False
```

Whether the model is *abstract*.

See also

`AbstractModel`

TransientModel

```
class odoo.models.TransientModel (env: Environment, ids: tuple[IdType, ...],
prefetch_ids: Reversible[IdType]) [source]
```

Model super-class for transient records, meant to be temporarily persistent, and regularly vacuum-cleaned.

A `TransientModel` has a simplified access rights management, all users can create new records, and may only access the records they created. The superuser has unrestricted access to all `TransientModel` records.

```
_transient_max_count = 0
```

maximum number of transient records, unlimited if `0`

```
_transient_max_hours = 1.0
```

maximum idle lifetime (in hours), unlimited if `0`

```
_transient_vacuum() [source]
```

Clean the transient records.

This unlinks old records from the transient model tables whenever the `_transient_max_count` or `_transient_max_hours` conditions (if any) are reached.

Actual cleaning will happen only once every 5 minutes. This means this method can be called frequently (e.g. whenever a new record is created).

between 5 and 10 minutes ago, the rest created/changed more than 12 minutes ago.

- age based vacuum will leave the 22 rows created/changed in the last 12 minutes
- count based vacuum will wipe out another 12 rows. Not just 2, otherwise each addition would immediately cause the maximum to be reached again.
- the 10 rows that have been created/changed the last 5 minutes will NOT be deleted

Fields

`class odoo.fields.Field` [\[source\]](#)

The field descriptor contains the field definition, and manages accesses and assignments of the corresponding field on records. The following attributes may be provided when instantiating a field:

Parameters:

string ([str](#)) – the label of the field seen by users; if not set, the ORM takes the field name in the class (capitalized).

help ([str](#)) – the tooltip of the field seen by users

readonly ([bool](#)) –

whether the field is readonly (default: `False`)

This only has an impact on the UI. Any field assignation in code will work (if the field is a stored field or an inversable one).

required ([bool](#)) – whether the value of the field is required (default: `False`)

index ([str](#)) –

whether the field is indexed in database, and the kind of index. Note: this has no effect on non-stored and virtual fields. The possible values are:

"btree" or `True` : standard index, good for many2one

"btree_not_null" : **BTREE index without NULL values (useful when most values are NULL, or when NULL is never searched for)**

"trigram" : Generalized Inverted Index (GIN) with trigrams (good for full-text search)

`None` or `False` : no index (default)

default (*value or callable*) – the default value for the field; this is either a static value, or a function taking a recordset and returning a value; use `default=None` to discard default values for the field

groups ([str](#)) – comma-separated list of group xml ids (string); this restricts the field access to the users of the given groups only

company_dependent ([bool](#)) –

whether the field value is dependent of the current company;

copy ([bool](#)) – whether the field value should be copied when the record is duplicated (default: True for normal fields, False for one2many and computed fields, including property fields and related fields)

store ([bool](#)) – whether the field is stored in database (default: True, False for computed fields)

default_export_compatible ([bool](#)) – whether the field must be exported by default in an import-compatible export

search ([str](#)) –

name of a method that implements search on the field. The method takes an operator and value. Basic domain optimizations are ran before calling this function. For instance, all '=' are transformed to 'in', and boolean fields conditions are made such that operator is 'in' / 'not in' and value is [True].

The method should return NotImplemented if it does not support the operator. In that case, the ORM can try to call it with other, semantically equivalent, operators. For instance, try with the positive operator if its corresponding negative operator is not implemented. The method must return a [Search domains](#) that replaces (field, operator, value) in its domain.

Note that a stored field can actually have a search method. The search method will be invoked to rewrite the condition. This may be useful for sanitizing the values used in the condition, for instance.

```
def _search_partner_ref(self, operator, value):
    if operator not in ('in', 'like'):
        return NotImplemented
    ... # add your logic here, example
    return Domain('partner_id.ref', operator, value)
```

Aggregation

Parameters:

aggregator ([str](#)) –

default aggregate function used by the webclient on this field when using "Group By" feature. Supported aggregators are:

count : number of rows

count_distinct : number of distinct rows

bool_and : true if all values are true, otherwise false

bool_or : true if at least one value is true, otherwise false

max : maximum value of all values

min : minimum value of all values

avg : the average (arithmetic mean) of all values

sum : sum of all values

```
@api.model
def _read_group_selection_field(self, values, domain):
    return ['choice1', 'choice2', ...] # available selection choices.

@api.model
def _read_group_many2one_field(self, records, domain):
    return records + self.search([custom_domain])
```

Computed Fields

Parameters:

compute ([str](#)) –

name of a method that computes the field

See also

[Advanced Fields/Compute fields](#)

precompute ([bool](#)) –

whether the field should be computed before record insertion in database. Should be used to specify manually some fields as `precompute=True` when the field can be computed before record insertion. (e.g. avoid statistics fields based on `search/_read_group`), many2one linking to the previous record, ... (default: `False`)

Warning

Precomputation only happens when no explicit value and no default value is provided to `create()`. This means that a default value disables the precomputation, even if the field is specified as `precompute=True`.

Precomputing a field can be counterproductive if the records of the given model are not created in batch. Consider the situation where many records are created one by one. If the field is not precomputed, it will normally be computed in batch at the `flush()`, and the prefetching mechanism will help making the computation efficient. On the other hand, if the field is precomputed, the computation will be made one by one, and will therefore not be able to take advantage of the prefetching mechanism.

Following the remark above, precomputed fields can be interesting on the lines of a one2many, which are usually created in batch by the ORM itself, provided that they are created by writing on the record that contains them.

compute_sudo ([bool](#)) – whether the field should be recomputed as superuser to bypass access rights (by default `True` for stored fields, `False` for non stored fields)

inverse ([str](#)) – name of a method that inverses the field (optional)

related ([str](#)) –
sequence of field names

See also

[Advanced fields/Related fields](#)

Basic Fields

class `odoo.fields.Boolean` [\[source\]](#)

Encapsulates a [bool](#).

class `odoo.fields.Char` [\[source\]](#)

Basic string field, can be length-limited, usually displayed as a single-line string in clients.

Parameters:

size ([int](#)) – the maximum size of values stored for that field

trim ([bool](#)) –

states whether the value is trimmed or not (by default, `True`). Note that the trim operation is applied by both the server code and the web client This ensures consistent behavior between imported data and UI-entered data.

The web client trims user input during in write/create flows in UI.

The server trims values during import (in `base_import`) to avoid discrepancies between trimmed form inputs and stored DB values.

translate ([bool](#) or *callable*) – enable the translation of the field's values; use `translate=True` to translate field values as a whole; `translate` may also be a callable such that

`translate(callback, value)` translates `value` by using `callback(term)` to retrieve the translation of terms.

class `odoo.fields.Float` [\[source\]](#)

Encapsulates a [float](#).

The precision digits are given by the (optional) `digits` attribute.

Parameters:

digits ([tuple](#) ([int](#), [int](#)) or [str](#)) – a pair (total, decimal) or a string referencing a `DecimalPrecision` record name.

min_display_digits ([int](#) or [str](#)) – An int or a string referencing a `DecimalPrecision` record name. Represents the minimum number of decimal digits to display in the UI. So if it's equal to 3: -

right tool to compare or round values with the correct precision.

The Float class provides some static methods for this purpose:

round() to round a float with the given precision. **is_zero()** to check if a float equals zero at the given precision. **compare()** to compare two floats at the given precision.

Example

To round a quantity with the precision of the unit of measure:

```
fields.Float.round(self.product_uom_qty, precision_rounding=self.product_uom_id.r
```

To check if the quantity is zero with the precision of the unit of measure:

```
fields.Float.is_zero(self.product_uom_qty, precision_rounding=self.product_uom_id
```

To compare two quantities:

```
field.Float.compare(self.product_uom_qty, self.qty_done, precision_rounding=self.
```

The compare helper uses the `__cmp__` semantics for historic purposes, therefore the proper, idiomatic way to use this helper is like so:

if `result == 0`, the first and second floats are equal
if `result < 0`, the first float is lower than the second
if `result > 0`, the first float is greater than the second

`class odoo.fields.Integer` [\[source\]](#) [↗](#)

Encapsulates an `int` [↗](#).

Advanced Fields

`class odoo.fields.Binary` [\[source\]](#) [↗](#)

Encapsulates a binary content (e.g. a file).

Parameters:

Encapsulates an html code content.

Parameters:

- sanitize** ([bool](#)) – whether value must be sanitized (default: `True`)
- sanitize_overridable** ([bool](#)) – whether the sanitation can be bypassed by the users part of the `base.group_sanitize_override` group (default: `False`)
- sanitize_tags** ([bool](#)) – whether to sanitize tags (only a white list of attributes is accepted, default: `True`)
- sanitize_attributes** ([bool](#)) – whether to sanitize attributes (only a white list of attributes is accepted, default: `True`)
- sanitize_style** ([bool](#)) – whether to sanitize style attributes (default: `False`)
- sanitize_conditional_comments** ([bool](#)) – whether to kill conditional comments. (default: `True`)
- sanitize_output_method** ([bool](#)) – whether to sanitize using html or xhtml (default: `html`)
- strip_style** ([bool](#)) – whether to strip style attributes (removed and therefore not sanitized, default: `False`)
- strip_classes** ([bool](#)) – whether to strip classes attributes (default: `False`)

`class odoo.fields.Image` [\[source\]](#)

Encapsulates an image, extending `Binary` .

If image size is greater than the `max_width` / `max_height` limit of pixels, the image will be resized to the limit by keeping aspect ratio.

Parameters:

- max_width** ([int](#)) – the maximum width of the image (default: `0` , no limit)
- max_height** ([int](#)) – the maximum height of the image (default: `0` , no limit)
- verify_resolution** ([bool](#)) – whether the image resolution should be verified to ensure it doesn't go over the maximum image resolution (default: `True`). See `odoo.tools.image.ImageProcess` for maximum image resolution (default: `50e6`).

Note

If no `max_width` / `max_height` is specified (or is set to 0) and `verify_resolution` is `False`, the field content won't be verified at all and a `Binary` field should be used.

`class odoo.fields.Monetary` [\[source\]](#)

Encapsulates a `float` expressed in a given `res_currency` .

currency_field ([str](#)) – name of the [many2one](#) field holding the res_currency this monetary field is expressed in (default: 'currency_id')

class `odoo.fields.Selection` [\[source\]](#)

Encapsulates an exclusive choice between different values.

Parameters:

selection ([list](#) ([tuple](#) ([str](#), [str](#))) or callable or [str](#)) – specifies the possible values for this field. It is given as either a list of pairs (value, label) , or a model method, or a method name.

selection_add ([list](#) ([tuple](#) ([str](#), [str](#)))) – provides an extension of the selection in the case of an overridden field. It is a list of pairs (value, label) or singletons (value,) , where singleton values must appear in the overridden selection. The new values are inserted in an order that is consistent with the overridden selection and this list:

```
selection = [('a', 'A'), ('b', 'B')]
selection_add = [('c', 'C'), ('b',)]
> result = [('a', 'A'), ('c', 'C'), ('b', 'B')]
```

ondelete –

provides a fallback mechanism for any overridden field with a selection_add. It is a dict that maps every option from the selection_add to a fallback action.

This fallback action will be applied to all records whose selection_add option maps to it.

The actions can be any of the following:

'set null' – the default, all records with this option will have their selection value set to False.

'cascade' – all records with this option will be deleted along with the option itself.

'set default' – all records with this option will be set to the default of the field definition

'set VALUE' – all records with this option will be set to the given value

<callable> – a callable whose first and only argument will be the set of records containing the specified Selection option, for custom processing

The attribute `selection` is mandatory except in the case of `related` or `extended` fields.

class `odoo.fields.Text` [\[source\]](#)

Very similar to [Char](#) but used for longer contents, does not have a size and usually displayed as a multiline text box.

Parameters:

translate ([bool](#) or callable) – enable the translation of the field's values; use `translate=True` to translate field values as a whole; `translate` may also be a callable such that

Date(time) Fields

Dates and **Datetimes** are very important fields in any kind of business application. Their misuse can create invisible yet painful bugs, this section aims to provide Odoo developers with the knowledge required to avoid misusing these fields.

When assigning a value to a Date/Datetime field, the following options are valid:

- A date or datetime object.
- A string in the proper server format:
 - YYYY-MM-DD for **Date** fields,
 - YYYY-MM-DD HH:MM:SS for **Datetime** fields.
- False OR None .

The Date and Datetime fields class have helper methods to attempt conversion into a compatible type:

- `to_date()` will convert to a `datetime.date` [↗](#)
- `to_datetime()` will convert to a `datetime.datetime` [↗](#).

Example

To parse date/datetimes coming from external sources:

```
fields.Date.to_date(self._context.get('date_from'))
```

Date / Datetime comparison best practices:

- Date fields can **only** be compared to date objects.
- Datetime fields can **only** be compared to datetime objects.

Warning

Strings representing dates and datetimes can be compared between each other, however the result may not be the expected result, as a datetime string will always be greater than a date string, therefore this practice is **heavily** discouraged.

Common operations with dates and datetimes such as addition, subtraction or fetching the start/end of a period are exposed through both **Date** and **Datetime** . These helpers are also available by importing `odoo.tools.date_utils` .

Note

independent from the timezone of the hosting server system. Timezone conversion is managed entirely by the client side.

`class odoo.fields.Date` [\[source\]](#)

Encapsulates a python `date` object.

`static today(*args) → datetime.date` [\[source\]](#)

Return the current day in the format expected by the ORM.

Note

This function may be used to compute default values.

`static context_today(record: BaseModel, timestamp: date | datetime | None = None) → date` [\[source\]](#)

Return the current date as seen in the client's timezone in a format fit for date fields.

Note

This method may be used to compute default values.

Parameters:

record – recordset from which the timezone will be obtained.

timestamp – optional datetime value to use instead of the current date and time (must be a datetime, regular dates can't be converted between timezones).

`static to_date(value) → datetime.date | None` [\[source\]](#)

Attempt to convert `value` to a `date` object.

Warning

If a datetime object is given as value, it will be converted to a date object and all datetime-specific information will be lost (HMS, TZ, ...).

Parameters:

value (`str` or `date` or `datetime`) – value to convert.

Returns:

an object representing `value`.

Parameters:

value – value to convert.

Returns:

a string representing `value` in the server's date format, if `value` is of type `datetime`, the hours, minute, seconds, tzinfo will be truncated.

static `start_of(value: D, granularity: Granularity) → D` [\[source\]](#)

Get start of a time period from a date or a datetime.

Parameters:

value – initial date or datetime.

granularity – type of period in string, can be year, quarter, month, week, day or hour.

Returns:

a date/datetime object corresponding to the start of the specified period.

static `end_of(value: D, granularity: Granularity) → D` [\[source\]](#)

Get end of a time period from a date or a datetime.

Parameters:

value – initial date or datetime.

granularity – Type of period in string, can be year, quarter, month, week, day or hour.

Returns:

A date/datetime object corresponding to the start of the specified period.

static `add(value: D, *args, **kwargs) → D` [\[source\]](#)

Return the sum of `value` and a `relativedelta`.

Parameters:

value – initial date or datetime.

args – positional args to pass directly to `relativedelta`.

kwargs – keyword args to pass directly to `relativedelta`.

Returns:

the resulting date/datetime.

static `subtract(value: D, *args, **kwargs) → D` [\[source\]](#)

Return the difference between `value` and a `relativedelta`.

Parameters:

Returns:

the resulting date/datetime.

`class odoo.fields.Datetime` [\[source\]](#)

Encapsulates a python `datetime` object.

`static now(*args) → datetime.datetime` [\[source\]](#)

Return the current day and time in the format expected by the ORM.

Note

This function may be used to compute default values.

`static today(*args) → datetime.datetime` [\[source\]](#)

Return the current day, at midnight (00:00:00).

`static context_timestamp(record: BaseModel, timestamp: datetime) → datetime` [\[source\]](#)

Return the given timestamp converted to the client's timezone.

Note

This method is *not* meant for use as a default initializer, because datetime fields are automatically converted upon display on client side. For default values, `now()` should be used instead.

Parameters:

record – recordset from which the timezone will be obtained.

timestamp (*datetime*) – naive datetime value (expressed in UTC) to be converted to the client timezone.

Returns:

timestamp converted to timezone-aware datetime in context timezone.

Return type:

datetime

`static to_datetime(value) → datetime.datetime | None` [\[source\]](#)

Convert an ORM `value` into a `datetime` value.

Parameters:

value (*str* or *date* or *datetime*) – value to convert.


```
static to_string(value: Union[datetime.datetime, Literal[False]])
```

→ Union[str, Literal[False]] [source]

Convert a `datetime` or `date` object to a string.

Parameters:

value (*datetime or date*) – value to convert.

Returns:

a string representing `value` in the server's datetime format, if `value` is of type `date`, the time portion will be midnight (00:00:00).

```
static start_of(value: D, granularity: Granularity) → D [source]
```

Get start of a time period from a date or a datetime.

Parameters:

value – initial date or datetime.

granularity – type of period in string, can be year, quarter, month, week, day or hour.

Returns:

a date/datetime object corresponding to the start of the specified period.

```
static end_of(value: D, granularity: Granularity) → D [source]
```

Get end of a time period from a date or a datetime.

Parameters:

value – initial date or datetime.

granularity – Type of period in string, can be year, quarter, month, week, day or hour.

Returns:

A date/datetime object corresponding to the start of the specified period.

```
static add(value: D, *args, **kwargs) → D [source]
```

Return the sum of `value` and a `relativedelta`.

Parameters:

value – initial date or datetime.

args – positional args to pass directly to `relativedelta`.

kwargs – keyword args to pass directly to `relativedelta`.

Returns:

the resulting date/datetime.

```
static subtract(value: D, *args, **kwargs) → D [source]
```

Return the difference between `value` and a `relativedelta`.

kwargs – keyword args to pass directly to `relativedelta` .

Returns:

the resulting date/datetime.

Relational Fields

`class odoo.fields.Many2one` [\[source\]](#)

The value of such a field is a recordset of size 0 (no record) or 1 (a single record).

Parameters:

comodel_name ([str](#)) – name of the target model Mandatory except for related or extended fields.

domain – an optional domain to set on candidate values on the client side (domain or a python expression that will be evaluated to provide domain)

context ([dict](#)) – an optional context to use on the client side when handling that field

ondelete ([str](#)) – what to do when the referred record is deleted; possible values are: 'set null' , 'restrict' , 'cascade'

bypass_search_access ([bool](#)) – whether access rights are bypassed on the comodel (default: False)

delegate ([bool](#)) – set it to True to make fields of the target model accessible from the current model (corresponds to `_inherits`)

check_company ([bool](#)) – Mark the field to be verified in `_check_company()` . Has a different behaviour depending on whether the field is company_dependent or not. Constrains non-company-dependent fields to target records whose company_id(s) are compatible with the record's company_id(s). Constrains company_dependent fields to target records whose company_id(s) are compatible with the currently active company.

`class odoo.fields.One2many` [\[source\]](#)

One2many field; the value of such a field is the recordset of all the records in `comodel_name` such that the field `inverse_name` is equal to the current record.

Parameters:

comodel_name ([str](#)) – name of the target model

inverse_name ([str](#)) – name of the inverse `Many2one` field in `comodel_name`

domain – an optional domain to set on candidate values on the client side (domain or a python expression that will be evaluated to provide domain)

context ([dict](#)) – an optional context to use on the client side when handling that field

bypass_search_access ([bool](#)) – whether access rights are bypassed on the comodel (default: False)

Many2many field; the value of such a field is the recordset.

Parameters:

comodel_name ([str](#)) – name of the target model (string) mandatory except in the case of related or extended fields

relation ([str](#)) – optional name of the table that stores the relation in the database

column1 ([str](#)) – optional name of the column referring to “these” records in the table `relation`

column2 ([str](#)) – optional name of the column referring to “those” records in the table `relation`

The attributes `relation`, `column1` and `column2` are optional. If not given, names are automatically generated from model names, provided `model_name` and `comodel_name` are different!

Note that having several fields with implicit relation parameters on a given model with the same `comodel` is not accepted by the ORM, since those field would use the same table. The ORM prevents two many2many fields to use the same relation parameters, except if

- both fields use the same model, `comodel`, and relation parameters are explicit; or
- at least one field belongs to a model with `_auto = False`.

Parameters:

domain – an optional domain to set on candidate values on the client side (domain or a python expression that will be evaluated to provide domain)

context ([dict](#)) – an optional context to use on the client side when handling that field

check_company ([bool](#)) – Mark the field to be verified in `_check_company()`. Add a default company domain depending on the field attributes.

`class odoo.fields.Command` [\[source\]](#)

`One2many` and `Many2many` fields expect a special command to manipulate the relation they implement.

Internally, each command is a 3-elements tuple where the first element is a mandatory integer that identifies the command, the second element is either the related record id to apply the command on (commands `update`, `delete`, `unlink` and `link`) either 0 (commands `create`, `clear` and `set`), the third element is either the `values` to write on the record (commands `create` and `update`) either the new `ids` list of related records (command `set`), either 0 (commands `delete`, `unlink`, `link`, and `clear`). This triplet is aliased as `CommandValue`.

Via Python, we encourage developers craft new commands via the various functions of this namespace. We also encourage developers to use the command identifier constant

identifier of the command.

CREATE = 0

UPDATE = 1

DELETE = 2

UNLINK = 3

LINK = 4

CLEAR = 5

SET = 6

classmethod `create(values: ValuesType) → CommandValue` [\[source\]](#)

Create new records in the comodel using `values`, link the created records to `self`.

In case of a **Many2many** relation, one unique new record is created in the comodel such that all records in `self` are linked to the new record.

In case of a **One2many** relation, one new record is created in the comodel for every record in `self` such that every record in `self` is linked to exactly one of the new records.

Return the command triple `(CREATE, 0, {values})`

classmethod `update(id: int, values: ValuesType) → CommandValue` [\[source\]](#)

Write `values` on the related record.

Return the command triple `(UPDATE, {id}, {values})`

classmethod `delete(id: int) → CommandValue` [\[source\]](#)

Remove the related record from the database and remove its relation with `self`.

In case of a **Many2many** relation, removing the record from the database may be prevented if it is still linked to other records.

Return the command triple `(DELETE, {id}, 0)`

classmethod `unlink(id: int) → CommandValue` [\[source\]](#)

inverse field is set as `onDelete='cascade'` . Otherwise, the value of the inverse field is set to False and the record is kept.

Return the command triple `(UNLINK, {id}, 0)`

classmethod `link(id: int) → CommandValue \[source\]`

Add a relation between `self` and the related record.

Return the command triple `(LINK, {id}, 0)`

classmethod `clear() → CommandValue \[source\]`

Remove all records from the relation with `self` . It behaves like executing the `unlink` command on every record.

Return the command triple `(CLEAR, 0, 0)`

classmethod `set(ids: Collection[int]) → CommandValue \[source\]`

Replace the current relations of `self` by the given ones. It behaves like executing the `unlink` command on every removed relation then executing the `link` command on every new relation.

Return the command triple `(SET, 0, {ids})`

Pseudo-relational fields

class `odoo.fields.Reference \[source\]`

Pseudo-relational field (no FK in database).

The field value is stored as a [string](#) following the pattern `"res_model,res_id"` in database.

class `odoo.fields.Many2oneReference \[source\]`

Pseudo-relational field (no FK in database).

The field value is stored as an [integer](#) id in database.

Contrary to [Reference](#) fields, the model has to be specified in a [Char](#) field, whose name has to be specified in the `model_field` attribute for the current [Many2oneReference](#) field.

Parameters:

model_field ([str](#)) – name of the [Char](#) where the model name is stored.

Computed Fields

```
from odoo import api
total = fields.Float(compute='_compute_total')

@api.depends('value', 'tax')
def _compute_total(self):
    for record in self:
        record.total = record.value + record.value * record.tax
```

- dependencies can be dotted paths when using sub-fields:

```
@api.depends('line_ids.value')
def _compute_total(self):
    for record in self:
        record.total = sum(line.value for line in record.line_ids)
```

- computed fields are not stored by default, they are computed and returned when requested. Setting `store=True` will store them in the database and automatically enable searching and grouping. Note that by default, `compute_sudo=True` is set on the field.
- searching on a computed field can also be enabled by setting the `search` parameter. The value is a method name returning a [Search domains](#).

```
upper_name = field.Char(compute='_compute_upper', search='_search_upper')

def _search_upper(self, operator, value):
    if operator == 'like':
        operator = 'ilike'
    return Domain('name', operator, value)
```

- computed fields are readonly by default. To allow *setting* values on a computed field, use the `inverse` parameter. It is the name of a function reversing the computation and setting the relevant fields:

```
document = fields.Char(compute='_get_document', inverse='_set_document')

def _get_document(self):
    for record in self:
        with open(record.get_document_path) as f:
            record.document = f.read()
```

```
f.write(record.document)
```

- multiple fields can be computed at the same time by the same method, just use the same method on all fields and set all of them:

```
discount_value = fields.Float(compute='_apply_discount')
total = fields.Float(compute='_apply_discount')

@api.depends('value', 'discount')
def _apply_discount(self):
    for record in self:
        # compute actual discount from discount percentage
        discount = record.value * record.discount
        record.discount_value = discount
        record.total = record.value - discount
```

Warning

While it is possible to use the same compute method for multiple fields, it is not recommended to do the same for the inverse method.

During the computation of the inverse, **all** fields that use said inverse are protected, meaning that they can't be computed, even if their value is not in the cache.

If any of those fields is accessed and its value is not in cache, the ORM will simply return a default value of `False` for these fields. This means that the value of the inverse fields (other than the one triggering the inverse method) may not give their correct value and this will probably break the expected behavior of the inverse method.

Related fields

A special case of computed fields are *related* (proxy) fields, which provide the value of a sub-field on the current record. They are defined by setting the `related` parameter and like regular computed fields they can be stored:

```
nickname = fields.Char(related='user_id.partner_id.name', store=True)
```

The value of a related field is given by following a sequence of relational fields and reading a field on the reached model. The complete sequence of fields to traverse is specified by the `related` attribute.

semantic-tree attributes are copied from the source field.

By default, related fields are:

- not stored
- not copied
- readonly
- computed in superuser mode

Add the attribute `store=True` to make it stored, just like computed fields. Related fields are automatically recomputed when their dependencies are modified.

Tip

You can specify precise field dependencies if you don't want the related field to be recomputed on any dependency change:

```
nickname = fields.Char(
    related='partner_id.name', store=True,
    depends=['partner_id'])
# The nickname will only be recomputed when the partner_id
# is modified, not when the name is modified on the partner.
```

Warning

You cannot chain `Many2many` or `One2many` fields in `related` fields dependencies.

`related` can be used to refer to a `One2many` or `Many2many` field on another model on the condition that it's done through a `Many2one` relation on the current model. `One2many` and `Many2many` are not supported and the results will not be aggregated correctly:

```
m2o_id = fields.Many2one()
m2m_ids = fields.Many2many()
o2m_ids = fields.One2many()

# Supported
d_ids = fields.Many2many(related="m2o_id.m2m_ids")
e_ids = fields.One2many(related="m2o_id.o2m_ids")

# Won't work: use a custom Many2many computed field instead
f_ids = fields.Many2many(related="m2m_ids.m2m_ids")
g_ids = fields.One2many(related="o2m_ids.o2m_ids")
```


If length of current recordset is 1, return id of unique record in it.

Raise an Error otherwise.

Model.display_name

Name **field** displayed by default in the web client

By default, it equals to **_rec_name** value field but the behavior can be customized by overriding **_compute_display_name**

Access Log fields

These fields are automatically set and updated if **_log_access** is enabled. It can be disabled to avoid creating or updating those fields on tables for which they are not useful.

By default, **_log_access** is set to the same value as **_auto**

Model.create_date

Stores when the record was created, **Datetime**

Model.create_uid

Stores *who* created the record, **Many2one** to a `res.users` .

Model.write_date

Stores when the record was last updated, **Datetime**

Model.write_uid

Stores who last updated the record, **Many2one** to a `res.users` .

Warning

_log_access *must* be enabled on **TransientModel** .

Reserved Field names

A few field names are reserved for pre-defined behaviors beyond that of automated fields. They should be defined on a model when the related behavior is desired:

Model.name

default value for **_rec_name** , used to display records in context where a representative "naming" is necessary.

toggles the global visibility of the record, if `active` is set to `False` the record is invisible in most searches and listing.

Boolean

Special methods:

`action_archive()` [\[source\]](#)

Set `active` to `False` on a recordset for active records.

Note, you probably want to override `write()` method if you want to take action once the active field changes.

`Model.action_unarchive()` [\[source\]](#)

Set `active` to `True` on a recordset for inactive records.

Note, you probably want to override `write()` method if you want to take action once the active field changes.

`Model.state`

lifecycle stages of the object, used by the `states` attribute on `fields`.

Selection

`Model.parent_id`

default_value of `_parent_name`, used to organize records in a tree structure and enables the `child_of` and `parent_of` operators in domains.

Many2one

`Model.parent_path`

When `_parent_store` is set to `True`, used to store a value reflecting the tree structure of `_parent_name`, and to optimize the operators `child_of` and `parent_of` in **Search domains**. It must be declared with `index=True` for proper operation.

Char

`Model.company_id`

Main field name used for Odoo multi-company behavior.

Used by `:meth:~odoo.models._check_company` to check multi company consistency. Defines whether a record is shared between companies (no value) or only accessible by the users of a given company.

Similarly to fields, you can declare `Constraint`, `Index` and `UniqueIndex`. The name of the attribute must begin with `_` to avoid name clashes with field names.

You can customize error messages. They can either be strings and their translation will be provided in the internal reflected constraint table. Otherwise, they can be functions that take `(env, diag)` as parameters which respectively denote the environment and psycopy diagnostics.

Example

```
class AModel(models.Model):
    _name = 'a.model'
    _my_check = models.Constraint("CHECK (x > y)", "x > y is not true")
    _name_idx = models.Index("(last_name, first_name)")
```

Recordsets

Interactions with models and records are performed through recordsets, an ordered collection of records of the same model.

Warning

Contrary to what the name implies, it is currently possible for recordsets to contain duplicates. This may change in the future.

Methods defined on a model are executed on a recordset, and their `self` is a recordset:

```
class AModel(models.Model):
    _name = 'a.model'
    def a_method(self):
        # self can be anything between 0 records and all records in the
        # database
        self.do_operation()
```

Iterating on a recordset will yield new sets of *a single record* ("singletons"), much like iterating on a Python string yields strings of a single characters:

```
print(record) # => a.model(1), then a.model(2), then a.model(3), ...
```

Field access

Recordsets provide an “Active Record” interface: model fields can be read and written directly from the record as attributes.

Note

When accessing non-relational fields on a recordset of potentially multiple records, use

`mapped()`:

```
total_qty = sum(self.mapped('qty'))
```

Field values can also be accessed like dict items, which is more elegant and safer than `getattr()` for dynamic field names. Setting a field’s value triggers an update to the database:

```
>>> record.name
Example Name
>>> record.company_id.name
Company Name
>>> record.name = "Bob"
>>> field = "name"
>>> record[field]
Bob
```

Warning

Trying to read a field on multiple records will raise an error for non relational fields.

Accessing a relational field (`Many2one` , `One2many` , `Many2many`) *always* returns a recordset, empty if the field is not set.

Record cache and prefetching

Odoo maintains a cache for the fields of the records, so that not every field access issues a database request, which would be terrible for performance. The following example queries the database only for the first statement:

To avoid reading one field on one record at a time, Odoo *prefetches* records and fields following some heuristics to get good performance. Once a field must be read on a given record, the ORM actually reads that field on a larger recordset, and stores the returned values in cache for later use. The prefetched recordset is usually the recordset from which the record comes by iteration. Moreover, all simple stored fields (boolean, integer, float, char, text, date, datetime, selection, many2one) are fetched altogether; they correspond to the columns of the model's table, and are fetched efficiently in the same query.

Consider the following example, where `partners` is a recordset of 1000 records. Without prefetching, the loop would make 2000 queries to the database. With prefetching, only one query is made:

```
for partner in partners:
    print partner.name           # first pass prefetches 'name' and 'lang'
                                # (and other fields) on all 'partners'
    print partner.lang
```

The prefetching also works on *secondary records*: when relational fields are read, their values (which are records) are subscribed for future prefetching. Accessing one of those secondary records prefetches all secondary records from the same model. This makes the following example generate only two queries, one for partners and one for countries:

```
countries = set()
for partner in partners:
    country = partner.country_id   # first pass prefetches all partners
    countries.add(country.name)    # first pass prefetches all countries
```

See also

The methods `search_fetch()` and `fetch()` can be used to populate the cache of records, typically in cases where the prefetching mechanism does not work well.

Method decorators

`odoo.api.autovacuum` (*method: C*) → C [\[source\]](#) [↗](#)

Decorate a method so that it is called by the daily vacuum cron job (model `ir.autovacuum`). This is typically used for garbage-collection-like tasks that do not deserve a specific

`odoo.api.constrains (*args) → Decorator` [\[source\]](#)

Decorate a constraint checker.

Each argument must be a field name used in the check:

```
@api.constrains('name', 'description')
def _check_description(self):
    for record in self:
        if record.name == record.description:
            raise ValidationError("Fields name and description must be different")
```

Invoked on the records on which one of the named fields has been modified.

Should raise `ValidationError` if the validation failed.

Warning

`@constrains` only supports simple field names, dotted names (fields of relational fields e.g. `partner_id.customer`) are not supported and will be ignored.

`@constrains` will be triggered only if the declared fields in the decorated method are included in the `create` or `write` call. It implies that fields not present in a view will not trigger a call during a record creation. A override of `create` is necessary to make sure a constraint will always be triggered (e.g. to test the absence of value).

One may also pass a single function as argument. In that case, the field names are given by calling the function with a model instance.

`odoo.api.depends (*args) → Decorator` [\[source\]](#)

Return a decorator that specifies the field dependencies of a “compute” method (for new-style function fields). Each argument must be a string that consists in a dot-separated sequence of field names:

```
pname = fields.Char(compute='_compute_pname')

@api.depends('partner_id.name', 'partner_id.is_company')
def _compute_pname(self):
    for record in self:
        if record.partner_id.is_company:
            record.pname = (record.partner_id.name or "").upper()
```

One may also pass a single function as argument. In that case, the dependencies are given by calling the function with the field's model.

odoo.api.depends_context (*args: [str](#)) → Decorator [\[source\]](#)

Return a decorator that specifies the context dependencies of a non-stored “compute” method. Each argument is a key in the context's dictionary:

```
price = fields.Float(compute='_compute_product_price')

@api.depends_context('pricelist')
def _compute_product_price(self):
    for product in self:
        if product.env.context.get('pricelist'):
            pricelist = self.env['product.pricelist'].browse(product.env.context['pricelist'])
        else:
            pricelist = self.env['product.pricelist'].get_default_pricelist()
        product.price = pricelist._get_products_price(product).get(product.id, 0.0)
```

All dependencies must be hashable. The following keys have special support:

- company (value in context or current company id),
- uid (current user id and superuser flag),
- active_test (value in env.context or value in field.context).

odoo.api.model (method: *C*) → *C* [\[source\]](#)

Decorate a record-style method where `self` is a recordset, but its contents is not relevant, only the model is. Such a method:

```
@api.model
def method(self, args):
    ...
```

odoo.api.model_create_multi (method: *Callable*[[*T*, [list](#)[*ValueType*]], *T*)

→ *Callable*[[*T*, [list](#)[*ValueType*] | *ValueType*, *T*] [\[source\]](#)

Decorate a method that takes a list of dictionaries and creates multiple records. The method may be called with either a single dict or a list of dicts:

```
record = model.create(vals)
```

Return a decorator to decorate an onchange method for given fields.

In the form views where the field appears, the method will be called when one of the given fields is modified. The method is invoked on a pseudo-record that contains the values present in the form. Field assignments on that record are automatically sent back to the client.

Each argument must be a field name:

```
@api.onchange('partner_id')
def _onchange_partner(self):
    self.message = "Dear %s" % (self.partner_id.name or "")
```

```
return {
    'warning': {'title': "Warning", 'message': "What is this?", 'type': 'notification'}
}
```

If the type is set to notification, the warning will be displayed in a notification. Otherwise it will be displayed in a dialog as default.

Warning

`@onchange` only supports simple field names, dotted names (fields of relational fields e.g. `partner_id.tz`) are not supported and will be ignored

Danger

Since `@onchange` returns a recordset of pseudo-records, calling any one of the CRUD methods (`create()`, `read()`, `write()`, `unlink()`) on the aforementioned recordset is undefined behaviour, as they potentially do not exist in the database yet.

Instead, simply set the record's field like shown in the example above or call the `update()` method.

Warning

It is not possible for a `one2many` or `many2many` field to modify itself via onchange. This is a webclient limitation - see [#2693](#).

The goal of this decorator is to allow client-side errors when unlinking records if, from a business point of view, it does not make sense to delete such records. For instance, a user should not be able to delete a validated sales order.

While this could be implemented by simply overriding the method `unlink` on the model, it has the drawback of not being compatible with module uninstallation. When uninstalling the module, the override could raise user errors, but we shouldn't care because the module is being uninstalled, and thus **all** records related to the module should be removed anyway.

This means that by overriding `unlink`, there is a big chance that some tables/records may remain as leftover data from the uninstalled module. This leaves the database in an inconsistent state. Moreover, there is a risk of conflicts if the module is ever reinstalled on that database.

Methods decorated with `@ondelete` should raise an error following some conditions, and by convention, the method should be named either `_unlink_if_<condition>` or `_unlink_except_<not_condition>`.

```
@api.ondelete(at_uninstall=False)
def _unlink_if_user_inactive(self):
    if any(user.active for user in self):
        raise UserError("Can't delete an active user!")

# same as above but with _unlink_except_* as method name
@api.ondelete(at_uninstall=False)
def _unlink_except_active_user(self):
    if any(user.active for user in self):
        raise UserError("Can't delete an active user!")
```

Parameters:

at_uninstall ([bool](#)) – Whether the decorated method should be called if the module that implements said method is being uninstalled. Should almost always be `False`, so that module uninstallation does not trigger those errors.

Danger

The parameter `at_uninstall` should only be set to `True` if the check you are implementing also applies when uninstalling the module.

For instance, it doesn't matter if when uninstalling `sale`, validated sales orders are being deleted because all data pertaining to `sale` should be deleted anyway, in that

behavior. In this case, `at_uninstall` should be set to `True`.

`odoo.api.private (method: C) → C` [\[source\]](#)

Decorate a record-style method to indicate that the method cannot be called using RPC. Example:

```
@api.private
def method(self, args):
    ...
```

If you have business methods that should not be called over RPC, you should prefix them with `"_"`. This decorator may be used in case of existing public methods that become non-RPC callable or for ORM methods.

Environment

`class odoo.api.Environment (cr: odoo.sql_db.BaseCursor, uid: int, context: dict, su: bool = False)` [\[source\]](#)

The environment stores various contextual data used by the ORM:

- `cr` : the current database cursor (for database queries);
- `uid` : the current user id (for access rights checks);
- `context` : the current context dictionary (arbitrary metadata);
- `su` : whether in superuser mode.

It provides access to the registry by implementing a mapping from model names to models. It also holds a cache for records, and a data structure to manage recomputations.

```
>>> records.env
<Environment object ...>
>>> records.env.uid
3
>>> records.env.user
res.user(3)
>>> records.env.cr
<Cursor object ...>
```

```
>>> self.env['res.partner']
res.partner()
>>> self.env['res.partner'].search([( 'is_company', '=', True), ( 'customer', '=', True)])
res.partner(7, 18, 12, 14, 17, 19, 8, 31, 26, 16, 13, 20, 30, 22, 29, 15, 23, 28, 74)
```

Some lazy properties are available to access the environment (contextual) data:

Environment.lang

Return the current language code.

Environment.user

Return the current user (as an instance).

Returns:

current user - sudoed

Return type:

res.users record

Environment.company

Return the current company (as an instance).

If not specified in the context (`allowed_company_ids`), fallback on current user main company.

Raises:

AccessError – invalid or unauthorized `allowed_company_ids` context key content.

Returns:

current company (default=`'self.user.company_id'`), with the current environment

Return type:

res.company record

Warning

No sanity checks applied in sudo mode! When in sudo mode, a user can access any company, even if not in his allowed companies.

This allows to trigger inter-company modifications, even if the current user doesn't have access to the targeted company.

Environment.companies

Raises:

AccessError – invalid or unauthorized `allowed_company_ids` context key content.

Returns:

current companies (default=``self.user.company_ids``), with the current environment

Return type:

`res.company` recordset

Warning

No sanity checks applied in sudo mode ! When in sudo mode, a user can access any company, even if not in his allowed companies.

This allows to trigger inter-company modifications, even if the current user doesn't have access to the targeted company.

Useful environment methods

`Environment.ref (xml_id: str, raise_if_not_found: Literal[True] = True) → BaseModel`
[\[source\]](#)

`Environment.ref (xml_id: str, raise_if_not_found: Literal[False]) → BaseModel | None`

Return the record corresponding to the given `xml_id`.

Parameters:

xml_id ([str](#)) – record `xml_id`, under the format `<module.id>`

raise_if_not_found ([bool](#)) – whether the method should raise if record is not found

Returns:

Found record or None

Raises:

ValueError – if record wasn't found and `raise_if_not_found` is True

`Environment.is_superuser () → bool \[source\]`

Return whether the environment is in superuser mode.

`Environment.is_admin () → bool \[source\]`

Return whether the current user has group "Access Rights", or is in superuser mode.

`Environment.is_system () → bool \[source\]`

Return whether the current user has group "Settings", or is in superuser mode.

`Environment.execute_query (query: odoo.tools.sql.SQL) → list [tuple] \[source\]`

Altering the environment

`Model.with_context (ctx: dict[str, typing.Any] | None = None, /, **overrides)
 → Self \[source\]`

Return a new version of this recordset attached to an extended context.

The extended context is either the provided `context` in which `overrides` are merged or the *current* context in which `overrides` are merged e.g.:

```
# current context is {'key1': True}
r2 = records.with_context({}, key2=True)
# -> r2.env.context is {'key2': True}
r2 = records.with_context(key2=True)
# -> r2.env.context is {'key1': True, 'key2': True}
```

`Model.with_user (user: BaseModel | IdType) → Self \[source\]`

Return a new version of this recordset attached to the given user, in non-superuser mode, unless `user` is the superuser (by convention, the superuser is always in superuser mode.)

`Model.with_company (company: BaseModel | IdType) → Self \[source\]`

Return a new version of this recordset with a modified context, such that:

```
result.env.company = company
result.env.companies = self.env.companies | company
```

Warning

When using an unauthorized company for current user, accessing the company(ies) on the environment may trigger an `AccessError` if not done in a sudoed environment.

`Model.with_env (env: Environment) → Self \[source\]`

Return a new version of this recordset attached to the provided environment.

Note

The returned recordset has the same prefetch object as `self`.

bypasses access rights checks.

Warning

Using `sudo` could cause data access to cross the boundaries of record rules, possibly mixing records that are meant to be isolated (e.g. records from different companies in multi-company environments).

It may lead to un-intuitive results in methods which select one record among many - for example getting the default company, or selecting a Bill of Materials.

Note

The returned recordset has the same prefetch object as `self`.

SQL Execution

The `cr` attribute on environments is the cursor for the current database transaction and allows executing SQL directly, either for queries which are difficult to express using the ORM (e.g. complex joins) or for performance reasons:

```
self.env.cr.execute("some_sql", params)
```

Warning

Executing raw SQL bypasses the ORM and, by consequent, Odoo security rules. Please make sure your queries are sanitized when using user input and prefer using ORM utilities if you don't really need to use SQL queries.

The recommended way to build SQL queries is to use the wrapper object

```
class odoo.tools.SQL (code: str | SQL = '', /, *args,
to_flush: Field | Iterable[Field] | None = None, **kwargs) [source]
```

An object that wraps SQL code with its parameters, like:

```
sql = SQL("UPDATE TABLE foo SET a = %s, b = %s", 'hello', 42)
cr.execute(sql)
```

escaped (as `%`), even if the code does not have parameters, like in `SQL("foo LIKE 'a%')"`.

The SQL wrapper is designed to be composable: the arguments can be either actual parameters, or SQL objects themselves:

```
sql = SQL(
    "UPDATE TABLE %s SET %s",
    SQL.identifier(tablename),
    SQL("%s = %s", SQL.identifier(columnname), value),
)
```

The combined SQL code is given by `sql.code`, while the corresponding combined parameters are given by the list `sql.params`. This allows to combine any number of SQL terms without having to separately combine their parameters, which can be tedious, bug-prone, and is the main downside of

`psycpg2.sql` <<https://www.psycpg.org/docs/sql.html>>.

The second purpose of the wrapper is to discourage SQL injections. Indeed, if `code` is a string literal (not a dynamic string), then the SQL object made with `code` is guaranteed to be safe, provided the SQL objects within its parameters are themselves safe.

The wrapper may also contain some metadata `to_flush`. If not `None`, its value is a field which the SQL code depends on. The metadata of a wrapper and its parts can be accessed by the iterator `sql.to_flush`.

`join(args: Iterable) → SQL [source]`

Join SQL objects or parameters with `self` as a separator.

`classmethod identifier(name: str, subname: str | None = None, to_flush: Field | None = None) → SQL [source]`

Return an SQL object that represents an identifier.

One important thing to know about models is that they don't necessarily perform database updates right away. Indeed, for performance reasons, the framework delays the recomputation of fields after modifying records. And some database updates are delayed, too. Therefore, before querying the database, one has to make sure that it contains the relevant data for the query. This operation is called *flushing* and performs the expected database updates.

Example

```
self.env.cr.execute(SQL("SELECT id FROM model WHERE partner_id IN %s", ids))
ids = [row[0] for row in self.env.cr.fetchall()]
```

Before every SQL query, one has to flush the data needed for that query. There are three levels for flushing, each with its own API. One can flush either everything, all the records of a model, or some specific records. Because delaying updates improves performance in general, we recommend to be *specific* when flushing.

Environment.flush_all () → [None](#) [\[source\]](#)

Flush all pending computations and updates to the database.

Model.flush_model (fnames: Collection[str] | None = None) → [None](#) [\[source\]](#)

Process the pending computations and database updates on `self`'s model. When the parameter is given, the method guarantees that at least the given fields are flushed to the database. More fields can be flushed, though.

Parameters:

fnames – optional iterable of field names to flush

Model.flush_recordset (fnames: Collection[str] | None = None) → [None](#) [\[source\]](#)

Process the pending computations and database updates on the records `self`. When the parameter is given, the method guarantees that at least the given fields on records `self` are flushed to the database. More fields and records can be flushed, though.

Parameters:

fnames – optional iterable of field names to flush

Because models use the same cursor and the **Environment** holds various caches, these caches must be invalidated when *altering* the database in raw SQL, or further uses of models may become incoherent. It is necessary to clear caches when using `CREATE`, `UPDATE` or `DELETE` in SQL, but not `SELECT` (which simply reads the database).

Example

```
# make sure 'state' is up-to-date in database
self.env['model'].flush_model(['state'])

self.env.cr.execute("UPDATE model SET state=%s WHERE state=%s", ['new', 'old'])
```


Just like flushing, one can invalidate either the whole cache, the cache of all the records of a model, or the cache of specific records. One can even invalidate specific fields on some records or all records of a model. As the cache improves performance in general, we recommend to be *specific* when invalidating.

`Environment.invalidate_all (flush: bool = True) → None [source]`

Invalidate the cache of all records.

Parameters:

flush – whether pending updates should be flushed before invalidation. It is `True` by default, which ensures cache consistency. Do not use this parameter unless you know what you are doing.

`Model.invalidate_model (fnames: Collection[str] | None = None, flush: bool = True) → None [source]`

Invalidate the cache of all records of `self`'s model, when the cached values no longer correspond to the database values. If the parameter is given, only the given fields are invalidated from cache.

Parameters:

fnames – optional iterable of field names to invalidate

flush – whether pending updates should be flushed before invalidation. It is `True` by default, which ensures cache consistency. Do not use this parameter unless you know what you are doing.

`Model.invalidate_recordset (fnames: Collection[str] | None = None, flush: bool = True) → None [source]`

Invalidate the cache of the records in `self`, when the cached values no longer correspond to the database values. If the parameter is given, only the given fields on `self` are invalidated from cache.

Parameters:

fnames – optional iterable of field names to invalidate

flush – whether pending updates should be flushed before invalidation. It is `True` by default, which ensures cache consistency. Do not use this parameter unless you know what you are doing.

The methods above keep the caches and the database consistent with each other. However, if computed field dependencies have been modified in the database, one has to inform the models for the computed fields to be recomputed. The only thing the framework needs to know is *what* fields have changed on *which* records.

```
self.env['model'].flush_model(['state'])

# use the RETURNING clause to retrieve which rows have changed
self.env.cr.execute("UPDATE model SET state=%s WHERE state=%s RETURNING id", ['new'
ids = [row[0] for row in self.env.cr.fetchall()]

# invalidate the cache, and notify the update to the framework
records = self.env['model'].browse(ids)
records.invalidate_recordset(['state'])
records.modified(['state'])
```

One has to figure out which records have been modified. There are many ways to do this, possibly involving extra SQL queries. In the example above, we take advantage of the `RETURNING` clause of PostgreSQL to retrieve the information without an extra query. After making the cache consistent by invalidation, invoke the method `modified` on the modified records with the fields that have been updated.

`Model.modified(fnames: Collection[str], create: bool = False, before: bool = False) → None [source]`

Notify that fields will be or have been modified on `self`. This invalidates the cache where necessary, and prepares the recomputation of dependent stored fields.

Parameters:

- fnames** – iterable of field names modified on records `self`
- create** – whether called in the context of record creation
- before** – whether called before modifying records `self`

Common ORM methods

Create/Update

`Model.create(vals_list: List[ValueType]) → Self [source]`

Create new records for the model.

The new records are initialized using the values from the list of dicts `vals_list`, and if necessary those from `default_get()`.

Parameters:

- vals_list** –

For backward compatibility, `vals_list` may be a dictionary. It is treated as a singleton list `[vals]`, and a single record is returned.

see `write()` for details

Returns:

the created records

Raises:

AccessError – if the current user is not allowed to create records of the specified model

ValidationError – if user tries to enter invalid value for a selection field

ValueError [↗](#) – if a field name specified in the create values does not exist.

UserError – if a loop would be created in a hierarchy of objects a result of the operation (such as setting an object as its own parent)

`Model.copy (default: ValuesType | None ↗ = None) → Self \[source\] ↗`

Duplicate record `self` updating it with default values.

Parameters:

default – dictionary of field values to override in the original values of the copied record, e.g:
`{'field_name': overridden_value, ...}`

Returns:

new records

`Model.default_get (fields: Sequence\[str\]↗) → ValuesType \[source\] ↗`

Return default values for the fields in `fields_list`. Default values are determined by the context, user defaults, user fallbacks and the model itself.

Parameters:

fields – names of field whose default is requested

Returns:

a dictionary mapping field names to their corresponding default values, if they have a default value.

Note

Unrequested defaults won't be considered, there is no need to return a value for fields whose names are not in `fields_list`.

`Model.name_create (name: str ↗) → Union[tuple↗\[int\]↗, str ↗], Literal[False]] \[source\] ↗`

The new record will be initialized with any default values applicable to this model, or provided through the context. The usual behavior of `create()` applies.

Parameters:

name – display name of the record to create

Returns:

the (id, display_name) pair value of the created record

`Model.write (vals: ValuesType) → typing.Literal[True]` [\[source\]](#) [↗](#)

Update all records in `self` with the provided values.

Parameters:

vals – fields to update and the value to set on them

Raises:

AccessError – if user is not allowed to modify the specified records/fields

ValidationError – if invalid values are specified for selection fields

UserError – if a loop would be created in a hierarchy of objects a result of the operation (such as setting an object as its own parent)

- For numeric fields (`Integer` , `Float`) the value should be of the corresponding type
- For `Boolean` , the value should be a `bool` [↗](#)
- For `Selection` , the value should match the selection values (generally `str` [↗](#), sometimes `int` [↗](#))
- For `Many2one` , the value should be the database identifier of the record to set
- The expected value of a `One2many` or `Many2many` relational field is a list of `Command` that manipulate the relation the implement. There are a total of 7 commands: `create()` , `update()` , `delete()` , `unlink()` , `link()` , `clear()` , and `set()` .
- For `Date` and `~odoo.fields.Datetime` , the value should be either a date(time), or a string.

Warning

If a string is provided for Date(time) fields, it must be UTC-only and formatted according to `odoo.tools.misc.DEFAULT_SERVER_DATE_FORMAT` and

`odoo.tools.misc.DEFAULT_SERVER_DATETIME_FORMAT`

- Other non-relational fields use a string for value

```
self.browse([7, 18, 12])
res.partner(7, 18, 12)
```

Model.search (*domain: DomainType*, *offset: int* = 0, *limit: int* | *None* = None, *order: str* | *None* = None) → Self [\[source\]](#)

Search for the records that satisfy the given *domain* **search domain**.

Parameters:

domain – **A search domain**. Use an empty list to match all records.

offset – number of results to ignore (default: none)

limit – maximum number of records to return (default: all)

order – sort string

Returns:

at most *limit* records matching the search criteria

Raises:

AccessError – if user is not allowed to access requested information

This is a high-level method, which should not be overridden. Its actual implementation is done by method `_search()` .

Model.search_count (*domain: DomainType*, *limit: int* | *None* = None) → int [\[source\]](#)

Return the number of records in the current model matching **the provided domain**.

Parameters:

domain – **A search domain**. Use an empty list to match all records.

limit – maximum number of record to count (upperbound) (default: all)

This is a high-level method, which should not be overridden. Its actual implementation is done by method `_search()` .

Model.search_fetch (*domain: DomainType*, *field_names: Sequence[str]* | *None* = None, *offset: int* = 0, *limit: int* | *None* = None, *order: str* | *None* = None) → Self [\[source\]](#)

Search for the records that satisfy the given *domain* **search domain**, and fetch the given fields to the cache. This method is like a combination of methods `search()` and `fetch()` , but it performs both tasks with a minimal number of SQL queries.

```
prefetch=True
```

offset – number of results to ignore (default: none)

limit – maximum number of records to return (default: all)

order – sort string

Returns:

at most `limit` records matching the search criteria

Raises:

AccessError – if user is not allowed to access requested information

```
Model.name_search (name: str = '', domain: DomainType | None = None,
operator: str = 'ilike', limit: int = 100) → list[tuple[int, str]]
[source]
```

Search for records that have a display name matching the given `name` pattern when compared with the given `operator`, while also matching the optional search domain (`domain`).

This is used for example to provide suggestions based on a partial value for a relational field. Should usually behave as the reverse of `display_name`, but that is not guaranteed.

This method is equivalent to calling `search()` with a search domain based on `display_name` and mapping `id` and `display_name` on the resulting search.

Parameters:

name – the name pattern to match

domain – search domain (see `search()` for syntax), specifying further restrictions

operator – domain operator for matching `name`, such as `'like'` or `'='`.

limit – max number of records to return

Returns:

list of pairs `(id, display_name)` for all matching records.

```
Model.fetch (field_names: Collection[str] | None = None) → None [source]
```

Make sure the given fields are in memory for the records in `self`, by fetching what is necessary from the database. Non-stored fields are mostly ignored, except for their stored dependencies. This method should be called to optimize code.

Parameters:

field_names – a collection of field names to fetch, or `None` for all accessible fields marked with `prefetch=True`

Raises:

AccessError – if user is not allowed to access requested information

→ [list](#) [ValueType] [\[source\]](#)

Read the requested fields for the records in `self`, and return their values as a list of dicts.

Parameters:

fields – field names to return (default is all fields)

load – loading mode, currently the only option is to set to `None` to avoid loading the `display_name` of m2o fields

Returns:

a list of dictionaries mapping field names to their values, with one dictionary per record

Raises:

AccessError – if user is not allowed to access requested information

ValueError – if a requested field does not exist

This is a high-level method that is not supposed to be overridden. In order to modify how fields are read from database, see methods `_fetch_query()` and `_read_format()`.

```
Model._read_group (domain: DomainType, groupby: Sequence[str] = (),
aggregates: Sequence[str] = (), having: DomainType = (), offset: int = 0,
limit: int | None = None, order: str | None = None) → list[tuple]
[source]
```

Get fields aggregations specified by `aggregates` grouped by the given `groupby` fields where record are filtered by the `domain`.

Parameters:

domain – [A search domain](#). Use an empty list to match all records.

groupby – list of groupby descriptions by which the records will be grouped. A groupby description is either a field (then it will be grouped by that field) or a string `'field:granularity'`. Right now, the only supported granularities are `'day'`, `'week'`, `'month'`, `'quarter'` OR `'year'`, and they only make sense for date/datetime fields. Additionally integer date parts are also supported: `'year_number'`, `'quarter_number'`, `'month_number'`, `'iso_week_number'`, `'day_of_year'`, `'day_of_month'`, `'day_of_week'`, `'hour_number'`, `'minute_number'` and `'second_number'`.

aggregates – list of aggregates specification. Each element is `'field:agg'` (aggregate field with aggregation function `'agg'`). The possible aggregation functions are the ones provided by [PostgreSQL](#), `'count_distinct'` with the expected meaning and `'recordset'` to act like `'array_agg'` converted into a recordset.

having – A domain where the valid “fields” are the aggregates.

offset – optional number of groups to skip

limit – optional max number of groups to return

`[(groupby_1_value, ... , aggregate_1_value_aggregate, ...), ...]` . If group is related field, the value of it will be a recordset (with a correct prefetch set).

Raises:

AccessError – if user is not allowed to access requested information

Fields

`Model.fields_get (allfields: Collection[str] | None = None, attributes: Collection[str] | None = None) → dict[str, ValueType] [source]`

Return the definition of each field.

The returned value is a dictionary (indexed by field name) of dictionaries. The `_inherits`'d fields are included. The string, help, and selection (if present) attributes are translated.

Parameters:

allfields – fields to document, all if empty or not provided

attributes – attributes to return for each field, all if empty or not provided

Returns:

dictionary mapping field names to a dictionary mapping attributes to values.

Search domains

A search domain is a first-order logical predicate used for filtering and searching recordsets. You combine simple conditions on a field expression with logical operators.

Domain can be used as a builder for domains.

```
# simple condition domains
d1 = Domain('name', '=', 'abc')
d2 = Domain('phone', 'like', '7620')

# combine domains
d3 = d1 & d2 # and
d4 = d1 | d2 # or
d5 = ~d1     # not

# combine and parse multiple domains (any iterable of domains)
Domain.AND([d1, d2, d3, ...])
Domain.OR([d4, d5, ...])

# constants
```


A domain can be a simple condition (field_expr, operator, value) where:

- **field_expr (str)**
a field name of the current model, or a relationship traversal through a [Many2one](#) using dot-notation e.g. 'street' or 'partner_id.country' . If the field is a date(time) field, you can also specify a part of the date using 'field_name.granularity' . The supported granularities are 'year_number' , 'quarter_number' , 'month_number' , 'iso_week_number' , 'day_of_week' , 'day_of_month' , 'day_of_year' , 'hour_number' , 'minute_number' , 'second_number' . They all use an integer as value.
- **operator (str)**
an operator used to compare the field_expr with the value . Valid operators are:
 - =**
equals to
 - !=**
not equals to
 - >**
greater than
 - >=**
greater than or equal to
 - <**
less than
 - <=**
less than or equal to
 - =?**
unset or equals to (returns true if value is either None or False , otherwise behaves like =)
 - =like (and not =like)**
matches field_expr against the value pattern. An underscore _ in the pattern stands for (matches) any single character; a percent sign % matches any string of zero or more characters.
 - like (and not like)**
matches field_expr against the %value% pattern. Similar to =like but wraps value with '%' before matching

case insensitive `=like`

in (and not in)

is equal to any of the items from `value`, `value` should be a collection of items

child_of

is a child (descendant) of a `value` record (value can be either one item or a list of items).

Takes the semantics of the model into account (i.e following the relationship field named by `_parent_name`).

parent_of

is a parent (ascendant) of a `value` record (value can be either one item or a list of items).

Takes the semantics of the model into account (i.e following the relationship field named by `_parent_name`).

any (and not any)

matches if any record in the relationship traversal through `field_expr` ([Many2one](#) , [One2many](#) , or [Many2many](#)) satisfies the provided domain `value` . The `field_expr` should be a field name.

any! (and not any!)

like `any` , but bypasses access checks.

- **value**

variable type, must be comparable (through `operator`) to the named field.

Example

To search for partners named *ABC*, with a phone or mobile number containing *7620*:

```
Domain('name', '=', 'ABC') & (
    Domain('phone', 'ilike', '7620') | Domain('mobile', 'ilike', '7620')
)
```

To search sales orders to invoice that have at least one line with a product that is out of stock:

To search for all partners born in the month of February:

```
Domain('birthday.month_number', '=', 2)
```

Domain can be used to serialize the domain as a `list` of simple conditions represented by 3-item `tuple` (or a `list`). Such a serialized form may be sometimes faster to read or write. Domain conditions can be combined using logical operators in a *prefix* notation. You can combine 2 domains using `'&'` (AND), `'|'` (OR) and you can negate 1 using `'!'` (NOT).

```
# parse a domain (from List to Domain)
domain = Domain([('name', '=', 'abc'), ('phone', 'like', '7620')])

# serialize domain as a List (from Domain to List)
domain_list = list(domain)
# will output:
# ['&', ('name', '=', 'abc'), ('phone', 'like', '7620')]
```

Domain.iter_conditions () → `Iterable[DomainCondition]` [\[source\]](#) [↗](#)

Yield simple conditions of the domain

Domain.map_conditions (function: Callable[[DomainCondition], Domain]) → `Domain` [\[source\]](#) [↗](#)

Map a function to each condition and return the combined result

Domain.optimize (model: BaseModel) → `Domain` [\[source\]](#) [↗](#)

Perform optimizations of the node given a model.

It is a pre-processing step to rewrite the domain into a logically equivalent domain that is a more canonical representation of the predicate. Multiple conditions can be merged together.

It applies basic optimizations only. Those are transaction-independent; they only depend on the model's fields definitions. No model-specific override is used, and the resulting domain may be reused in another transaction without semantic impact. The model's fields are used to validate conditions and apply type-dependent optimizations. This optimization level may be useful to simplify a domain that is sent to the client-side, thereby reducing its payload/complexity.

Dynamic time values

In the context of search domains, for **date and datetime fields**, the value can be a moment relative to *now* in the timezone of the user. A simple language is provided to specify these dates. It is a space-separated string of terms. The first term is optional and is "today" (at midnight) or "now". Then, each term starts with "+" (add), "-" (subtract) or "=" (set), followed by an integer and date unit or a lower-case weekday.

The date units are: "d" (days), "w" (weeks), "m" (months), "y" (years), "H" (hours), "M" (minutes), "S" (seconds). For weekdays, "+" and "-" mean next and previous weekday (unless we are already in that weekday) and "=" means in current week starting on Monday. When setting a date, the lower-units (hours, minutes and seconds) are set to 0.

Example

```
Domain('some_date', '<', 'now') # now
Domain('some_date', '<', 'today') # today at midnight
Domain('some_date', '<', '-3d +1H') # now - 3 days + 1 hour
Domain('some_date', '<', '=3H') # today at 3:00:00
Domain('some_date', '<', '=5d') # 5th day of current month at midnight
Domain('some_date', '<', '=1m') # January, same day of month at midnight
Domain('some_date', '>=', '=monday -1w') # Monday of the previous week
```

Unlink

`Model.unlink ()` → `Literal[True]` [\[source\]](#)

Delete the records in `self`.

Raises:

- AccessError** – if the user is not allowed to delete all the given records
- UserError** – if the record is default property for other records

Record(set) information

`Model.ids`

Return the list of actual record ids corresponding to `self`.

`odoo.models.env`

Returns the environment of the given recordset.

Type:

The subset of records in `self` that exist. It can be used as a test on records:

```
if record.exists():
    ...
```

By convention, new records are returned as existing.

`Model.ensure_one ()` → `Self` [\[source\]](#)

Verify that the current recordset holds a single record.

Raises:

`odoo.exceptions.ValueError` – `len(self) != 1`

`Model.get_metadata ()` → `list` [\[ValuesType\]](#) [\[source\]](#)

Return some metadata about the given records.

Returns:

list of ownership dictionaries for each requested record with the following keys:

`id`: object id

`create_uid`: user who created the record

`create_date`: date when the record was created

`write_uid`: last user who changed the record

`write_date`: date of the last change to the record

`xmlid`: XML ID to use to refer to this record (if there is one), in format `module.name`

`xmlids`: list of dict with `xmlid` in format `module.name`, and `noupdate` as boolean

`noupdate`: A boolean telling if the record will be updated or not

Operations

Recordsets are immutable, but sets of the same model can be combined using various set operations, returning new recordsets.

- `record in set` returns whether `record` (which must be a 1-element recordset) is present in `set`. `record not in set` is the inverse operation
- `set1 <= set2` and `set1 < set2` return whether `set1` is a subset of `set2` (resp. strict)
- `set1 >= set2` and `set1 > set2` return whether `set1` is a superset of `set2` (resp. strict)
- `set1 | set2` returns the union of the two recordsets, a new recordset containing all records present in either source

Recordsets are iterable so the usual Python tools are available for transformation (`map()` [↗](#), `sorted()` [↗](#), `ifilter()` , ...) however these return either a `list` [↗](#) or an `iterator` [↗](#), removing the ability to call methods on their result, or to use set operations.

Recordsets therefore provide the following operations returning recordsets themselves (when possible):

Filter

`Model.filtered (func: str ↗ | Callable[[Self], bool ↗] | Domain) → Self \[source\] ↗`

Return the records in `self` satisfying `func` .

Parameters:

func – a function, Domain or a dot-separated sequence of field names

Returns:

recordset of records satisfying `func`, may be empty.

```
# only keep records whose company is the current user's
records.filtered(lambda r: r.company_id == user.company_id)

# only keep records whose partner is a company
records.filtered("partner_id.is_company")
```

`Model.filtered_domain (domain: DomainType) → Self \[source\] ↗`

Return the records in `self` satisfying the domain and keeping the same order.

Parameters:

domain – [A search domain](#).

Map

`Model.mapped (func: str ↗ | Callable[[Self], T]) → list ↗ | BaseModel \[source\] ↗`

Apply `func` on all records in `self` , and return the result as a list or a recordset (if `func` return recordsets). In the latter case, the order of the returned recordset is arbitrary.

Parameters:

func – a function or a dot-separated sequence of field names

Returns:

self if `func` is falsy, result of `func` applied to all `self` records.

The provided function can be a string to get field values:

```
# returns a List of names
records.mapped('name')

# returns a recordset of partners
records.mapped('partner_id')

# returns the union of all partner banks, with duplicates removed
records.mapped('partner_id.bank_ids')
```

Note

Since V13, multi-relational field access is supported and works like a mapped call:

```
records.partner_id # == records.mapped('partner_id')
records.partner_id.bank_ids # == records.mapped('partner_id.bank_ids')
records.partner_id.mapped('name') # == records.mapped('partner_id.name')
```

Sort

`Model.sorted (key: Callable[[Self], typing.Any] | str | None = None, reverse: bool = False) → Self \[source\]`

Return the recordset `self` ordered by `key`.

Parameters:

key –

It can be either of:

a function of one argument that returns a comparison key for each record

a string representing a comma-separated list of field names with optional NULLS (FIRST|LAST), and (ASC|DESC) directions

`None`, in which case records are ordered according the default model's order

reverse – if `True`, return the result in reverse order

```
# sort records by name
records.sorted(key=lambda r: r.name)
# sort records by name in descending order, then by id
records.sorted('name DESC, id')
```

Grouping

`Model.grouped (key: str | Callable\[\[Self\], T\]) → dict[typing.Any, Self]
\[source\]`

Eagerly groups the records of `self` by the `key`, returning a dict from the `key`'s result to recordsets. All the resulting recordsets are guaranteed to be part of the same prefetch-set.

Provides a convenience method to partition existing recordsets without the overhead of a `_read_group()`, but performs no aggregation.

Note

unlike `itertools.groupby()`, does not care about input ordering, however the tradeoff is that it can not be lazy

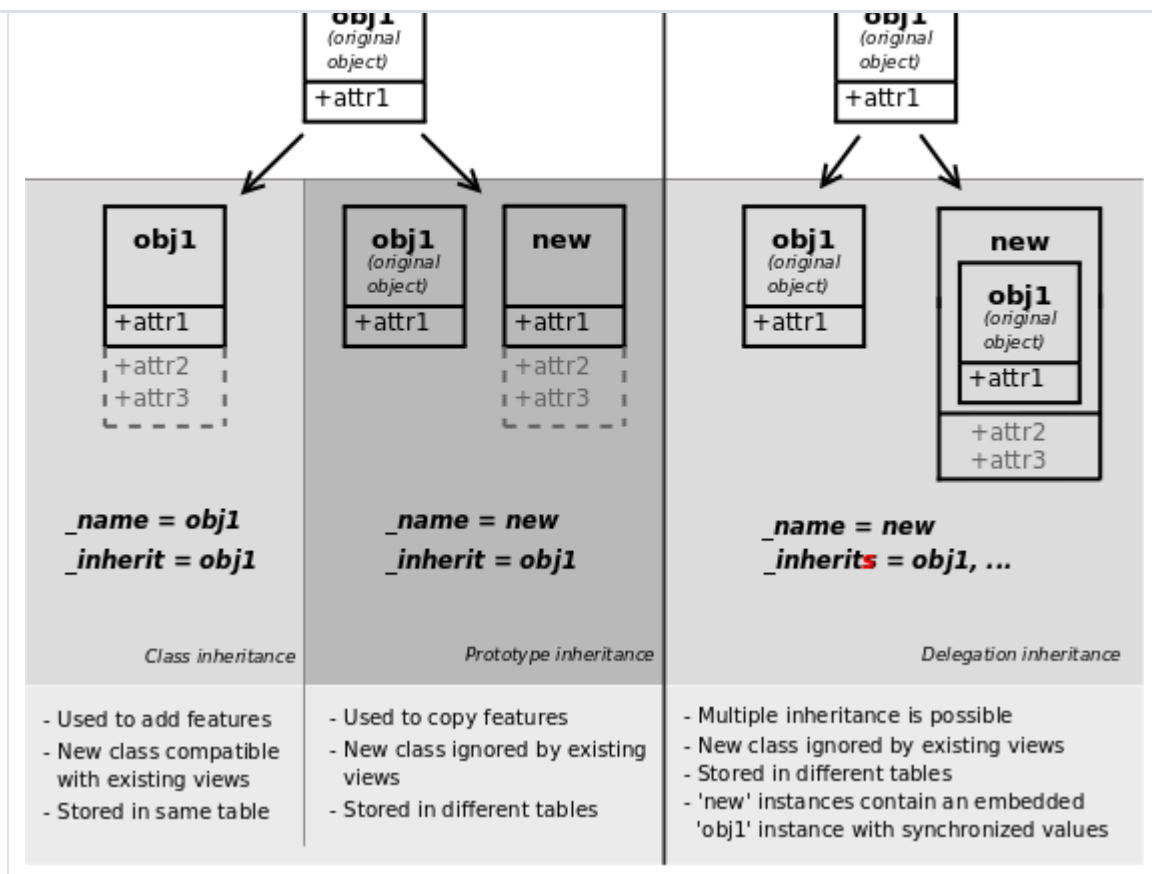
Parameters:

key – either a callable from a `Model` to a (hashable) value, or a field name. In the latter case, it is equivalent to `itemgetter(key)` (aka the named field's value)

Inheritance and extension

Odoo provides three different mechanisms to extend models in a modular way:

- creating a new model from an existing one, adding new information to the copy but leaving the original module as-is
- extending models defined in other modules in-place, replacing the previous version
- delegating some of the model's fields to records it contains



Classical inheritance

When using the `_inherit` and `_name` attributes together, Odoo creates a new model using the existing one (provided via `_inherit`) as a base. The new model gets all the fields, methods and meta-information (defaults & al) from its base.

```
class Inheritance0(models.Model):
    _name = 'inheritance.0'
    _description = 'Inheritance Zero'

    name = fields.Char()

    def call(self):
        return self.check("model 0")

    def check(self, s):
        return "This is {} record {}".format(s, self.name)

class Inheritance1(models.Model):
    _name = 'inheritance.1'
    _inherit = ['inheritance.0']
    _description = 'Inheritance One'
```

and using them:

```
a = env['inheritance.0'].create({'name': 'A'})
b = env['inheritance.1'].create({'name': 'B'})

a.call()
b.call()
```

will yield:

"This is model 0 record A" "This is model 1 record B"

the second model has inherited from the first model's `check` method and its `name` field, but overridden the `call` method, as when using standard [Python inheritance](#).

Extension

When using `_inherit` but leaving out `_name`, the new model replaces the existing one, essentially extending it in-place. This is useful to add new fields or methods to existing models (created in other modules), or to customize or reconfigure them (e.g. to change their default sort order)

```
class Extension0(models.Model):
    _name = 'extension.0'
    _description = 'Extension zero'

    name = fields.Char(default="A")

class Extension0(models.Model):
    _inherit = 'extension.0'

    description = fields.Char(default="Extended")
```

```
record = env['extension.0'].create({})
record.read()[0]
```

will yield:

```
{'name': "A", 'description': "Extended"}
```

explicitly set.

Note

It will also yield the various **automatic fields** unless they've been disabled

Delegation

The third inheritance mechanism provides more flexibility (it can be altered at runtime) but less power: using the `_inherits` a model *delegates* the lookup of any field not found on the current model to "children" models. The delegation is performed via **Reference** fields automatically set up on the parent model.

The main difference is in the meaning. When using Delegation, the model **has one** instead of **is one**, turning the relationship in a composition instead of inheritance

```
class Screen(models.Model):
    _name = 'delegation.screen'
    _description = 'Screen'

    size = fields.Float(string='Screen Size in inches')

class Keyboard(models.Model):
    _name = 'delegation.keyboard'
    _description = 'Keyboard'

    layout = fields.Char(string='Layout')

class Laptop(models.Model):
    _name = 'delegation.laptop'
    _description = 'Laptop'

    _inherits = {
        'delegation.screen': 'screen_id',
        'delegation.keyboard': 'keyboard_id',
    }

    name = fields.Char(string='Name')
    maker = fields.Char(string='Maker')

    # a Laptop has a screen
    screen_id = fields.Many2one('delegation.screen', required=True, ondelete="cascade")
```

```
record = env['delegation.laptop'].create({
    'screen_id': env['delegation.screen'].create({'size': 13.0}).id,
    'keyboard_id': env['delegation.keyboard'].create({'layout': 'QWERTY'}).id,
})
record.size
record.layout
```

will result in:

```
13.0
'QWERTY'
```

and it's possible to write directly on the delegated field:

```
record.write({'size': 14.0})
```

Warning

when using delegation inheritance, methods are *not* inherited, only fields

Warning

- `_inherits` is more or less implemented, avoid it if you can;
- chained `_inherits` is essentially not implemented, we cannot guarantee anything on the final behavior.

Fields Incremental Definition

A field is defined as class attribute on a model class. If the model is extended, one can also extend the field definition by redefining a field with the same name and same type on the subclass. In that case, the attributes of the field are taken from the parent class and overridden by the ones given in subclasses.

For instance, the second class below only adds a tooltip on the field `state`

```
class FirstFoo(models.Model):
    state = fields.Selection(..., required=True)
```

```
class WrongFirstFooClassName(models.Model):
    _name = 'first.foo' # force the model name
    _inherit = ['first.foo']
    state = fields.Selection(help="Blah blah blah")
```

Error management

The Odoo Exceptions module defines a few core exception types.

Those types are understood by the RPC layer. Any other exception type bubbling until the RPC layer will be treated as a 'Server error'.

exception `odoo.exceptions.UserError (message)` [\[source\]](#)

Generic error managed by the client.

Typically when the user tries to do something that has no sense given the current state of a record.

exception `odoo.exceptions.RedirectWarning (message, action, button_text, additional_context=None)` [\[source\]](#)

Warning with a possibility to redirect the user instead of simply displaying the warning message.

Parameters:

message ([str](#)) – exception message and frontend modal content

action_id ([int](#)) – id of the action where to perform the redirection

button_text ([str](#)) – text to put on the button that will trigger the redirection.

additional_context ([dict](#)) – parameter passed to action_id. Can be used to limit a view to active_ids for example.

exception `odoo.exceptions.AccessDenied (message='Access Denied')` [\[source\]](#)

Login/password error.

Note

Traceback only visible in the logs.

Example

Remove the traceback, cause and context of the exception, hiding where the exception occurred but keeping the exception message.

This method must be called in all situations where we are about to print this exception to the users.

It is OK to leave the traceback (thus to *not* call this method) if the exception is only logged in the logs, as they are only accessible by the system administrators.

exception `odoo.exceptions.AccessError (message)` [\[source\]](#) 

Access rights error.

Example

When you try to read a record that you are not allowed to.

exception `odoo.exceptions.CacheMiss (record, field)` [\[source\]](#) 

Missing value(s) in cache.

Example

When you try to read a value in a flushed cache.

exception `odoo.exceptions.MissingError (message)` [\[source\]](#) 

Missing record(s).

Example

When you try to write on a deleted record.

exception `odoo.exceptions.ValidationError (message)` [\[source\]](#) 

Violation of python constraints.

Example

When you try to create a new user with a login which already exist in the db.

Contact Support

Ask the Odoo Community

